

# AI / ML Integration

Prompt engineering, Gemini AI integration and parsing

## AI/ML Integration Documentation

### Project: FitBuddy — AI-Powered Personal Fitness Coach

#### Overview

FitBuddy leverages **Google Gemini 2.0 Flash** (a state-of-the-art large language model) to power all three core AI scenarios. The integration uses the official [google-generativeai](#) Python SDK.

#### 1. AI Model Used

| Parameter | Value                                          |
|-----------|------------------------------------------------|
| Model     | gemini-2.0-flash                               |
| Provider  | Google DeepMind                                |
| SDK       | google-generativeai v0.8.3                     |
| API Type  | REST-based generative AI                       |
| Access    | Google AI Studio API Key (free tier available) |

#### Why Gemini 2.0 Flash?

- Fast response times (typically 8-15 seconds for a full 7-day plan)
- Strong instruction-following for structured JSON output

- ■ Free tier available at [aistudio.google.com](https://aistudio.google.com)
- ■ Supports long-context prompts with user profile data
- ■ Reliable JSON schema adherence

---

## 2. Prompt Engineering

### Strategy: Role-Based System Prompts with JSON Schema

All prompts follow this structure:

1. **Role definition** — Tell the AI who it is
2. **User profile data** — Personalization context
3. **Feedback section** — (Only for Scenario 2) Additional constraints
4. **Strict output schema** — JSON structure the AI must return
5. **Output instruction** — "Return ONLY valid JSON, no markdown"

---

### Scenario 1: Workout Plan Generation Prompt

```
prompt = f""" You are FitBuddy, an expert personal fitness coach. Generate a detailed, personalized 7-day workout plan. User Profile: - Name: {profile['name']} - Age: {profile['age']} years - Weight: {profile['weight']} kg - Fitness Goal: {profile['goal'].replace('_', ' ').title()} - Workout Intensity: {profile['intensity'].title()} Return ONLY valid JSON in this exact format (no markdown, no explanation): {{ "plan title": "string", "goal summary": "string (2-3 sentences)", "weekly_overview": "string (brief overview)", "days": [ {{ "day": "Day 1 - Monday", "focus": "string (e.g. Upper Body Strength)", "duration_minutes": 45, "warmup": "string", "exercises": [ {{ "name": "string", "sets": 3, "reps": "string", "rest": "string", "tips": "string" }} ], "cooldown": "string", "motivational_note": "string" }} ], "weekly_tips": ["tip1", "tip2", "tip3"] }} """
```

#### Key Design Decisions:

- `profile['goal'].replace('_', ' ').title()` — Humanizes the goal name for the AI
- Explicit JSON schema prevents hallucination of extra fields
- `"no markdown, no explanation"` critical for parsing reliability

---

### Scenario 2: Feedback-Based Plan Update

The same prompt as Scenario 1 is used, but with an additional section:

```
feedback_section = f"\nUser Feedback/Modifications: {feedback}" if feedback else ""
```

This appended to the prompt means the AI sees:

- The original user profile

- The desired modifications
- And must reconcile both in the new plan

---

### Scenario 3: Nutrition Tip Prompt

```
prompt = f""" You are FitBuddy, a certified nutritionist. Give a concise, actionable nutrition and recovery tip for someone named {name} whose fitness goal is "{goal.replace('_', ' ')}". Format your response as JSON: {"tip_title": "string", "tip": "string (3-4 sentences)", "foods_to_include": ["food1", "food2", "food3", "food4"], "foods_to_avoid": ["food1", "food2", "food3"], "hydration_advice": "string", "recovery_advice": "string"} Return ONLY valid JSON, no markdown. """
```

---

## 3. Response Parsing & Error Handling

### Challenge: AI Returns Markdown Code Fences

Problem: Even when instructed not to, Gemini sometimes wraps responses in:

```
```json { ... } ```
```

### Solution: Code Fence Stripping

```
text = response.text.strip() if text.startswith("```"): text = text.split("```")[1] if text.startswith("json"): text = text[4:] text = text.strip() return json.loads(text)
```

This ensures reliable JSON extraction regardless of AI formatting behavior.

### Error Handling

```
try: plan = generate_workout_plan(profile.model_dump()) except json.JSONDecodeError as e: raise HTTPException(status_code=500, detail=f"AI response parsing error: {str(e)}") except Exception as e: raise HTTPException(status_code=500, detail=str(e))
```

---

## 4. Context Management

### User Profile as Context

The full user profile (name, age, weight, goal, intensity) is injected into every prompt, ensuring the AI's output is truly personalized and not generic.

## Feedback as Additional Context

When updating a plan, the AI receives the complete original user profile **plus** the feedback. This allows it to:

- Maintain profile-appropriate exercise difficulty
- Apply the specific change requested
- Keep the 7-day structure intact

## 5. Temperature & Token Settings

Default settings are used from the Gemini SDK (`gemini-2.0-flash`):

- **Temperature**: ~1.0 (creative but structured)
- **Max Output Tokens**: Model default (sufficient for 7-day JSON plans)
- **Top-P / Top-K**: Default values

These defaults work well for this use case as the JSON schema provides sufficient structure constraints.

## 6. Performance Metrics (Observed)

| Operation                 | Typical Duration |
|---------------------------|------------------|
| Workout Plan Generation   | 10–20 seconds    |
| Plan Update with Feedback | 10–18 seconds    |
| Nutrition Tip Generation  | 5–10 seconds     |

## 7. Security

- API key stored in **environment variable** (`GEMINI_API_KEY`)
- Key never exposed in responses or logs
- Fallback to a default key for development convenience only
- In production, the fallback should be removed

## 8. Future AI Enhancements

| Enhancement         | Description                                          |
|---------------------|------------------------------------------------------|
| Streaming Responses | Stream plan generation word-by-word for better UX    |
| Model Fine-tuning   | Fine-tune on fitness domain data for better accuracy |
| Memory / Context    | Store conversation history for multi-turn refinement |
| Image Understanding | Accept user body photos for better personalization   |
| Safety Filters      | Add content safety layers for medical disclaimers    |